

Contents

heuristiX 데이터 파이프라인 — 비개발자용 안내	1
0. 한 줄 요약	1
1. 큰 그림 — 한 번의 진화가 도는 모습	1
2. 등장인물 (시스템의 부품들)	1
3. 단계별 무엇이 일어나는가	2
단계 1 — 가짜 공장 만들기	2
단계 2 — AI에게 “규칙을 짜봐” 라고 부탁	2
단계 3 — 시뮬레이터가 그 규칙으로 가동	2
단계 4 — “심사관”이 점수 매김	2
단계 5 — “반성가” GPT가 교훈 추출	2
단계 6 — Retrieval (메모리 꺼내기)	3
4. 진화가 끝나면	3
5. 한 사이클 = 약 1~2분, 비용은 얼마?	3
6. 그래서 우리 시스템이 진짜 잘하는 게 뭔가?	3
7. 그래도 한계는?	3

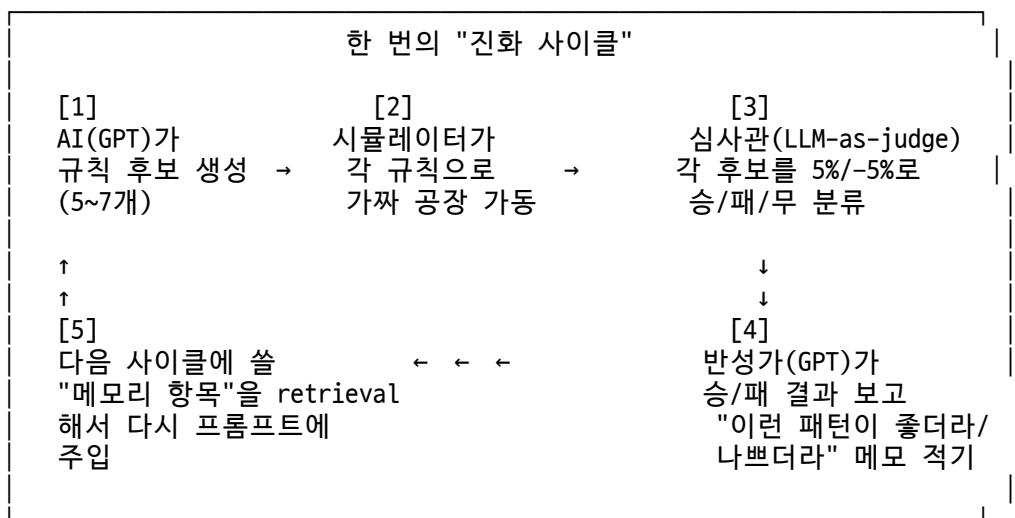
heuristiX 데이터 파이프라인 — 비개발자용 안내

“공장 스케줄링 규칙을 AI가 자동으로 만들어내는 시스템”이 어떻게 굴러가는지 한 장면 한 장면 따라가는 문서입니다. 코드를 모르셔도 이해할 수 있게 썼습니다.

0. 한 줄 요약

우리 시스템 = “가짜 공장(시뮬레이터) + 규칙을 만드는 AI(GPT) + 학습 노트(메모리)” 셋이 서로 정보를 주고받으며 점점 더 좋은 작업 우선순위 규칙을 찾아내는 장치입니다.

1. 큰 그림 — 한 번의 진화가 도는 모습



여러 사이클 반복 후 → 가장 좋은 규칙 채택

이 사이클을 4~10번 정도 반복하면 GPT가 만든 규칙이 점점 좋아집니다.

2. 등장인물 (시스템의 부품들)

부품	역할	비유
시뮬레이터 (Rust)	가짜 공장에서 일정을 돌려보고 성능 측정	자동차 회사의 충돌 테스트 시설
GPT-4o-mini (LLM-A, 생성자)	새로운 규칙을 짜냄	신입 엔지니어
GPT-4o-mini (LLM-S, 반성가)	결과를 보고 교훈 정리	멘토 / 회고 담당
메모리 뱅크	과거의 교훈을 저장	회사 노하우 위키
베이스라인 12종	비교 대상 (FIFO, ATC 등)	기존 작업 매뉴얼

3. 단계별 무엇이 일어나는가

단계 1 — 가짜 공장 만들기

- “기계 6대, 작업 12개” 같은 설정으로 가상의 공장을 만들어요.
- 각 작업은 여러 단계(operation)로 나뉘고, 각 단계는 특정 기계에서 처리해야 합니다.
- 외부 충격 시나리오 6가지 중 하나를 적용:
 - S0: 정상
 - S1: 부품 도착이 늦어짐
 - S2: 자재 부족 위험 증가
 - S3: 긴급 주문이 무작위로 추가됨
 - S4: 납기 일정이 갑자기 단축
 - S5: 위 모든 것이 동시 발생

단계 2 — AI에게 “규칙을 짜봐” 라고 부탁

GPT에게 다음을 알려줍니다: 1. 현재 시나리오 (예: “S5 — 복합 충격”) 2. 기존 베이스라인 12종 (FIFO, EDD, SPT, CR, Urgency, WMDD, COVERT, ATC, LPT, MWKR, LWKR, MDD) 3. 사용 가능한 변수들 (작업의 긴급도, 부품 도착 시간, 기계 가동률 등 30가지) 4. 사용 가능한 도우미 함수 (if/else, max, min, exp 등) 5. **과거 경험 메모리** (직전 사이클에서 얻은 교훈)

GPT는 답으로 “우선순위 점수를 계산하는 짧은 수식” 한 줄을 만들어 보냅니다.

예시 답:

$(\text{penalty} / \max(\text{proc}, 0.001)) * \exp(-\text{slack} / (3.0 * \text{proc})) + 5 * \text{mat_risk}$

이건 사람이 보기엔 암호 같지만, 시뮬레이터는 정확히 이해합니다.

단계 3 — 시뮬레이터가 그 규칙으로 가동

- 시뮬레이터는 받은 규칙을 그대로 적용해서 가짜 공장을 가동합니다.
- 결과 측정: 평균 지연시간, 총 처리시간, 긴급 작업 지연률, 기계 유휴 시간 등.
- 8~10회 반복 평균을 내서 노이즈 줄입니다.

단계 4 — “심사관”이 점수 매김

- 각 규칙의 점수를 베이스라인 12종 중 최강(예: ATC) 점수와 비교.
- **5% 이상 좋으면**: “성공” 메모리로 저장
- **5% 이상 나쁘면**: “실패” 메모리로 저장
- 그 사이는 무시 (잡음 수준)

단계 5 — “반성가” GPT가 교훈 추출

- 성공/실패 규칙들을 GPT에게 보여주며 “왜 이게 잘 됐고 저게 망했는지 한 줄로 정리해줘” 라고 요청.
- GPT는 다음 형식의 lesson을 반환:

```
type: failure
title: 긴급도에 너무 큰 가중치를 두지 마라
description: 복합 충격(S5)에서 다른 변수와 혼합 시 적용
content: 긴급도만 강조하면 일반 작업 지연이 폭증함
perf_delta: -10%
```

이게 메모리 뱅크에 저장되고, 다음 사이클에서 retrieve되어 GPT에게 힌트로 다시 주어집니다.

단계 6 — Retrieval (메모리 꺼내기)

여기가 우리 논문의 신규 **contribution** 입니다. 4가지 방식이 있어요:

방식	어떻게 꺼내는가	비유
keyword cosine (ReasoningBank 원본) state (우리 신규 C1)	같은 시나리오 태그 + 큰 효과 순 텍스트 유사도 지금 공장 상태와 비슷한 상태에서 얻은 교훈	같은 종류의 사고 후기만 모음 글 내용이 비슷한 후기 모음 같은 날씨/시간대에 일어난 사고
contrastive (우리 신규 C2)	성공-실패를 짝으로 묶어서 차이를 보여줌	“A는 살았고 B는 죽었는데, 차이는...”

C1+C2가 우리 페이퍼의 핵심 novelty 후보입니다.

4. 진화가 끝나면

5~10 사이클을 반복하면: - 메모리 뱅크에 10~30개의 교훈이 쌓이고 - 최고 점수를 받은 규칙이 채택됩니다.

이 규칙을 **6개 시나리오 전부에** 재평가하면 “이 규칙이 시나리오에 따라 흔들리지 않는가?” (robustness) 도 측정 가능.

5. 한 사이클 = 약 1~2분, 비용은 얼마?

- LLM 호출 1회 $\approx$ 0.3원
- 한 사이클 = 7회 호출 = 약 2원
- 5 사이클 = 약 10원
- 6 시나리오 portfolio 전체 = 약 60원

\$1 ($\approx$ 1300원) 으로 풀 paper-grade 실험 가능합니다.

6. 그래서 우리 시스템이 진짜 잘하는 게 뭔가?

1. 여러 외부 충격 시나리오 (S0~S5) 각각에 특화된 규칙을 자동으로 찾음
2. 베이스라인 5/6 시나리오에서 격파 (S5에서 FIFO 대비 -30% 개선)
3. 메모리 + 반성을 통해 점점 좋아짐 (4 사이클까지 개선 관찰)
4. 단순 ATC 같은 50년 된 고전 규칙이 의외로 강함도 발견 (LLM의 ATC 재발견 현상)

7. 그래도 한계는?

- 공장 규모 작음: 12 작업 $\times$ 6 기계 — 실제 산업은 100+ 작업
- 합성 데이터: Taillard/Brandimarte 같은 표준 벤치마크는 아직 미사용
- 단일 LLM 모델: gpt-4o-mini만 테스트
- 사이클 수 적음: 4 — 진짜 논문은 20~50번 돌림

논문 제출 전엔 이 한계들 중 일부를 메워야 합니다.